# CSE470: Software Engineering

## Software Requirements Specification (SRS)

End-of-Semester Submission Template

# FleetPulse

Smart Fleet Logistics & Delivery Management System


Prepared By

| Student ID | Name |
|---|---|
| 24101618 | Ram Narayan Bachhar Ayosh |
| 22299427 | Abdullah Al Kawsar |
| 24101352 | Mohammad Faiyaz Mazumder |
| 24101629 | MH Sakline Shafiq |


Project Information

| | |
|---|---|
| Section | 11 |
| Section Faculty | Nazmus Sakib Touhid |
| GitHub Repository Link | https://github.com/Ayosh-Bachhar/FleetPulse-Smart-Fleet-Logistics-and-Delivery-Management-System |
| Tech Stack Used | React with Vite, FastAPI (Python), MongoDB, Beanie ODM, Leaflet, JWT and bcrypt |

# 1. Introduction

## 1.1 Purpose

This document sets out what FleetPulse must do and how its features will work. FleetPulse is a web application for managing a delivery company's vehicles, drivers, orders and trips. It will help staff plan deliveries, calculate charges, track simulated vehicle movement and record proof of delivery.

## 1.2 Scope

FleetPulse is designed for Admins, Dispatchers, Drivers and Customers. Its 20 features cover the work from receiving an order to completing a delivery and reviewing the results. Customers can follow their orders through their accounts or share a public tracking link. The project does not include physical GPS devices, online payments or a separate mobile app.

# 2. Overall Description

## 2.1 Product Perspective

FleetPulse will work as a standalone system for a delivery company. Staff will use it to organize daily deliveries, while customers will use it to place and track orders. The React interface will connect to FastAPI, and Beanie ODM will handle data stored in MongoDB. External APIs will provide holiday, weather and route data and send staff alerts.

## 2.2 Product High-Level Summary

Fleet and workforce: Staff can manage vehicles, drivers and shifts. They can also check when a vehicle needs servicing and review its fuel use and mileage.
Orders and pricing: Customers can place orders, and dispatchers can place orders for them. The system will check loads, calculate charges, control order status changes and store delivery proof.

Dispatch and routing: Dispatchers can group orders into trips and assign a driver and vehicle. The system will arrange stops, calculate route details and show simulated progress with an estimated arrival time.

Visibility and reporting: Users can track deliveries and receive status updates. Each role will have a dashboard, and staff can export reports on delays and fuel costs.

## 2.3 Users and Characteristics

**Admin:** manages vehicles and drivers and reviews the company's delivery statistics and reports.

**Dispatcher:** places orders for customers, schedules drivers and vehicles, plans trips and follows delivery progress.

**Driver:** views assigned deliveries and uploads a photo as proof of delivery.

**Customer:** creates an account, places orders and follows their own deliveries.

**Public tracking visitor:** opens a valid UUID link to see an order's status and estimated arrival time

without logging in.

### 2.4 Operating Environment

FleetPulse will run in a modern web browser on a computer. The backend needs Python and a connection to MongoDB. React and Vite are used to build the frontend. Live API calls and map tiles need internet access. Saved API responses and fallback data will keep the demonstration usable if an API is unavailable.

### 2.5 Design and Implementation Constraints

This project follows the constraints defined in the CSE470 Project Outline, including mandatory MVC architecture and the approved tech-stack policy. State your specific choices below.

We will use React with Vite, FastAPI, MongoDB and Beanie ODM. The MVC structure will use Beanie documents as Models, React pages and components as Views, and FastAPI routers as Controllers. Business rules will be kept in services/. All database access will go through Beanie; raw queries will not be used. The project will not use MERN or a plain HTML/CSS-only frontend.

The application has no planned AI-powered feature. ODM use is allowed, and Calendarific can be used on its free plan. Members may share an API, although our plan assigns a different API to each member. We will not use Nager. Each member will build five features, including their frontend, backend and database work. Authentication is required but does not count toward the 20 features. The main branch must run from a fresh clone, with merges at least once a week.

### 2.6 Assumptions and Dependencies (Optional)

Vehicle movement will be simulated. The team will need free access to Calendarific, OpenWeather, GeoApify and Discord, along with the required keys or webhook settings. API responses will be saved for reuse, with fallback data available during outages.

## 3. System Architecture

### 3.1 Architectural Overview

Requests follow this path: React View → FastAPI Controller → service layer → Beanie Model → MongoDB. The Controller checks the request and the user's permissions. Services apply the business rules and use Models to read or save data. Results return through the Controller to the View. API adapters fetch external data, and a scheduled task updates simulated vehicle positions through the same services and Models.

### 3.2 Technology Stack

Frontend: React with Vite; Leaflet for maps.

Backend: FastAPI with Python routers, service modules and role checks.

Database: MongoDB, with all access handled by Beanie ODM.

Authentication: JWT in HTTP-only cookies and bcrypt password hashing.

External services: Calendarific, OpenWeather, GeoApify and Discord webhook.

### 3.3 Class Diagram

[Draw the Class Diagram of your project showing all the classes and relationships necessary to implement your system]

## 4. Requirements and Features

List every requirement and its associated feature(s) below. Per the Project Outline (Section 3.2.1), all CRUD operations on the same entity by the same actor using the same logic count as ONE feature — do not split them. A minimum of 20 features is required.

| Req. ID | Requirement | Feature Description | Actor | MVC Components Touched | Sprint Completed | Status |
|---|---|---|---|---|---|---|
| R1 | Manage vehicles and check availability. | F1: Fleet management. Add, view, update and search vehicle records, including capacity, fuel type and status. Only available vehicles can be dispatched. | Admin, Dispatcher | Model: Vehicle; View: FleetPage; Controller: VehicleRouter | Pending | Planned |
| R2 | Manage drivers and check their license classes. | F2: Driver management. Keep driver records and license classes. Match drivers only with vehicle types their licenses allow. | Admin, Dispatcher | Model: Driver, User; View: DriverPage; Controller: DriverRouter | Pending | Planned |

| | | | | | | |
|---|---|---|---|---|---|---|
| R3 | Schedule drivers and vehicles without overlaps. | F3: Shift roster. Assign a driver and vehicle to a time slot. Reject overlapping assignments and use Calendarific to warn when a shift falls on a public holiday. | Admin, Dispatcher | Model: Shift, Driver, Vehicle, ApiCache; View: ShiftPage; Controller: ShiftRouter | Pending | Planned |
| R4 | Keep vehicles due for service off delivery trips. | F4: Maintenance alerts. Flag a vehicle for service after the set number of simulated kilometres. Block dispatch until an authorized staff member clears it for use. | Admin, Dispatcher | Model: Vehicle, MaintenanceRecord; View: MaintenancePage; Controller: MaintenanceRouter | Pending | Planned |
| R5 | Record fuel use and mileage for each trip. | F5: Fuel and mileage log. Record each trip's fuel use and mileage. Calculate fuel efficiency and flag unusual consumption using the agreed limits. | Driver, Dispatcher, Admin | Model: FuelLog, Trip, Vehicle; View: FuelLogPage; Controller: FuelRouter | Pending | Planned |

| | | | | | | |
|---|---|---|---|---|---|---|
| R6 | Let customers and dispatchers place delivery orders. | F6: Order intake. Customers can place orders, and dispatchers can place them on a customer's behalf. Check weight, size, pickup and delivery addresses, and the rush flag. Customers can see only their own orders. | Customer, Dispatcher | Model: Order, User; View: OrderForm, MyOrders; Controller: OrderRouter | Pending | Planned |
| R7 | Keep loads within the vehicle's capacity. | F7: Capacity check. Add up the weight and volume of the orders assigned to a vehicle. Reject the load if either total exceeds its limit. Run this check before dispatch. | Dispatcher | Model: Order, Vehicle, Trip; View: LoadCheckPanel; Controller: CapacityRouter | Pending | Planned |

| | | | | | | |
|---|---|---|---|---|---|---|
| R8 | Calculate the delivery charge. | F8: Pricing engine. Calculate distance × rate + weight charge + rush fee + weather surcharge. Use OpenWeather for weather data and show how the total was calculated. | Customer, Dispatcher | Model: Order, PricingRules, ApiCache; View: PriceBreakdown; Controller: PricingRouter | Pending | Planned |
| R9 | Allow only valid order status changes. | F9: Order lifecycle. Track each order through Placed, Warehouse, Out for delivery and Delivered. Reject invalid status changes and updates made without permission. | Dispatcher, Driver, Customer (view) | Model: Order; View: OrderStatusView; Controller: OrderStatusRouter | Pending | Planned |
| R10 | Record proof when a delivery is completed. | F10: Delivery proof. The assigned driver uploads a photo for an order that is out for delivery. Once the upload is accepted, | Driver | Model: DeliveryProof, Order; View: DeliveryProofForm; Controller: ProofRouter | Pending | Planned |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | mark the order as Delivered. | | | | |
| R11 | Assign orders, a driver and a vehicle to a trip. | F11: Dispatch planning. Group orders into a trip and assign a driver and vehicle. Check capacity, license, shift, maintenance and availability before dispatch. Reject conflicting assignments. | Dispatcher | Model: Trip, Order, Vehicle, Driver, Shift; View: DispatchPage; Controller: DispatchRouter | Pending | Planned |
| R12 | Get route distance and travel time. | F12: Route distance and time. Use GeoApify to get the trip's distance and travel time. Save the results for reuse in trip planning and tracking. | Dispatcher | Model: Route, Trip, ApiCache; View: RouteSummary; Controller: RouteRouter | Pending | Planned |

| | | | | | | |
|---|---|---|---|---|---|---|
| R13 | Arrange the stops for a trip. | F13: Stop sequencing. Use our own nearest-neighbour algorithm to arrange the stops. Keep each stop linked to its order and respect the agreed pickup and delivery rules. | Dispatcher | Model: Route, Trip, Order; View: StopSequenceView; Controller: SequenceRouter | Pending | Planned |
| R14 | Simulate vehicle movement along the route. | F14: Simulated GPS. Move the simulated vehicle along its route every five seconds through a scheduled backend task. Save its position and stop movement when the trip ends. | Dispatcher, Driver (view) | Model: PositionUpdate, Trip, Route; View: SimulationView; Controller: PositionRouter | Pending | Planned |

| | | | | | | |
|---|---|---|---|---|---|---|
| R15 | Estimate arrival times and flag delays. | F15: ETA prediction. Estimate arrival using the remaining route distance and average speed. Flag delays against the planned arrival threshold and handle a speed of zero without a calculation error. | Dispatcher, Driver, Customer (view) | Model: PositionUpdate, Route, Trip; View: ETAView; Controller: ETARouter | Pending | Planned |
| R16 | Show trips and vehicle positions on a map. | F16: Map view. Show stops, the route and the simulated vehicle position on a Leaflet map. Highlight trips flagged as delayed by the ETA service. | Admin, Dispatcher, Driver | Model: Trip, Route, PositionUpdate; View: TrackingMap; Controller: MapRouter | Pending | Planned |

| | | | | | | |
|---|---|---|---|---|---|---|
| R17 | Let customers track their deliveries. | F17: Public tracking page. Give each order a UUID tracking link. Anyone with the link can see its status and ETA without logging in. Customers can also view their own orders in their accounts. | Customer, Public tracking visitor | Model: Order, Trip, PositionUpdate; View: PublicTrackingPage, MyOrders; Controller: TrackingRouter | Pending | Planned |
| R18 | Notify users when an order's status changes. | F18: Notifications. Notify the relevant customer, driver and dispatcher whenever an order's status changes. Show an unread count in the app and send a staff alert through the Discord webhook. | Customer, Driver, Dispatcher | Model: Notification, NotificationEvent, Order; View: NotificationBell; Controller: NotificationRouter | Pending | Planned |

| | | | | | | |
|---|---|---|---|---|---|---|
| R19 | Show useful statistics for each user role. | F19: Role dashboards. Give Admins, Dispatchers, Drivers and Customers statistics relevant to their work. Use only the records each role is allowed to access. | Admin, Dispatcher, Driver, Customer | Model: User, Order, Trip, Vehicle, FuelLog; View: RoleDashboard; Controller: DashboardRouter | Pending | Planned |
| R20 | Create and export delay and fuel-cost reports. | F20: Reports and exports. Create reports on delivery delays and fuel costs from stored trips and fuel records. Let authorized staff export the same data as CSV. | Admin, Dispatcher | Model: Trip, PositionUpdate, FuelLog; View: ReportsPage; Controller: ReportRouter | Pending | Planned |

### 4.1 Feature Count Summary

| | |
|---|---|
| Minimum features required | 20 |
| Total features listed in Section 4 above | 20 |
| Total features fully completed (“Done” per Section 7.2 of the Project Outline) | 0 |

## 5. External Interface Requirements (Optional)

| **Interface Type** | **Description** |
|---|---|
| User Interface | React pages with forms, delivery status, Leaflet maps, notifications |

| | |
|---|---|
| | and CSV downloads. |
| Hardware Interface | No dedicated hardware is needed. GPS movement is simulated, and drivers upload photos from their devices. |
| Software Interface (APIs, third-party services) | Calendarific supplies holidays, OpenWeather supplies weather data, and GeoApify supplies route details. Discord sends staff alerts. Beanie ODM handles MongoDB access. |
| Communication Interface | The frontend and backend exchange JSON over HTTP. Photos use multipart uploads, and reports download as CSV files. API keys stay on the server. |

## 6. Non-Functional Requirements (Optional)

| **Category** | **Requirement** |
|---|---|
| Performance | |
| Security | |
| Usability | |
| Reliability / Availability | |
| Scalability | |

## 7. Sprint History and Review Ceremony Log

Record the goals and attainment shared during each weekly sprint review ceremony, per Section 7.1 of the Project Outline.

| **Sprint** | **Goals Set (Start of Week)** | **Attainment Reported (End of Week)** | **Features Completed This Sprint** |
|---|---|---|---|
| Sprint 1 | Proposed: shared setup and authentication; F1, F2, F6, F9. | | |
| Sprint 2 | Proposed: F3, F4, F7, F8, F11, F12. | | |
| Sprint 3 | Proposed: F5, F10, F13, F14, F15, F18 . | | |

| | | | |
|---|---|---|---|
| Sprint 4 | Proposed: F16, F17, F19, F20; integration and testing. | | |

### 7.1 Deviations and Lessons Learned

[Briefly explain any sprint where attainment did not match the planned goals, why it happened, and what was adjusted for the following sprint.]

## 8. Testing Summary

[Briefly describe how each major feature was tested (manual testing steps, sample test cases, or automated tests). A short table or list is acceptable.]

## 9. AI Tool Usage Disclosure

Per the Project Outline's Academic Integrity & AI Usage Policy (Section 6), disclose every instance where a generative AI tool was used during development. Leave the table empty (do not delete it) if no AI tool was used.

| Feature / Task | AI Tool Used | How It Was Used (be specific) | I Can Fully Explain This Code (Yes/No) |
|---|---|---|---|
| | | | |
| | | | |
| | | | |

[By submitting this document, you confirm that you are able to personally explain, modify, and debug every piece of code listed above during your viva.]

## 10. Individual Contribution Summary

**Ram Narayan Bachhar Ayosh (24101618):** Will develop F1 Fleet management, F2 Driver management, F3 Shift roster; F4 Maintenance alerts; F5 Fuel and mileage log. Other assigned work, excluded from the feature count: registration, initial repository structure, Beanie setup and sample fleet data.

**Abdullah Al Kawsar (22299427):** Will develop F6 Order intake; F7 Capacity check; F8 Pricing engine; F9 Order lifecycle; F10 Delivery proof. Other assigned work, excluded from the feature count: login, shared access checks, input validation and sample order data.

**Mohammad Faiyaz Mazumder (24101352):** Will develop F11 Dispatch planning; F12 Route distance

and time; F13 Stop sequencing; F14 Simulated GPS; F15 ETA prediction. Other assigned work, excluded from the feature count: logout, simulation scheduler setup and checking that a fresh clone runs correctly.

**MH Sakline Shafiq (24101629):** Will develop F16 Map view; F17 Public tracking page; F18 Notifications; F19 Role dashboards; F20 Reports and export. Other assigned work, excluded from the feature count: password reset, shared navigation, README coordination and organizing test results.

Each member will build the frontend, backend and database parts of their five assigned features. We will agree on shared requirements and how the modules connect, then test them together. Actual hours spent and problems solved by each member will be recorded as the work is completed.

## 11. Pre-Submission Checklist

| **Checklist Item** | **Confirmed (Yes/No)** |
|---|---|
| The application implements the MVC architecture | Yes |
| At least 20 features are listed and each is correctly counted per the granularity rule | Yes |
| No forbidden framework (e.g. Django, Flask as primary backend) was used | Yes |
| All four sprints' goals and attainment are recorded in Section 11 | |
| The GitHub repository is up to date and the section faculty is added as a collaborator | Yes |
| The AI Usage Disclosure table (Section 9) is complete and honest | Yes |
| I can personally explain and defend every feature listed in this document during the viva | |